HW1 Reflection: Building an AI Research Assistant with a Coding Agent

CS59300 AI-Assisted Software Engineering · Fall 2026

Contents

1. Which coding agent I used, and how 1
2. Where the agent was particularly useful 1
3. Where the agent failed, and how I corrected it 2
4. How much I inspected, modified, or debugged 2
5. What I learned about controlling a coding agent 3

1. Which coding agent I used, and how

The primary agent was **omp (oh-my-pi)**, an agentic coding CLI on the Bun runtime with a skill system (`SKILL.md` discovery, `/skill:<name>` injection), Rust-native helpers (`@oh-my-pi/pi-natives`: PDF, grep, vectors), and MCP tool routing (Exa search included). I used it end-to-end rather than for a single skeleton dump: architecture planning (Astro vs Next.js vs single-file MVP), project scaffolding (`server.ts` + `lib/` + `index.html`), API integration (Exa/arXiv/Semantic Scholar/ OpenAlex), PDF pipeline work (pure-TS probe → Pi native `pdfToMarkdown`), SQLite schema + FTS5, SSE streaming UI, debugging (six recorded UI review loops v1–v6), and deployment verification (`bun run server.ts` → `:4321`). Supplemental public skills informed the literature workflow: `flyer-Li/paper-analyst` (structured 6-section output, anti-hallucination tags), `tronghieu/deep-reader` (Keshav three-pass, Adler synthesis), and the official `exa-search` skill. Three project-local skills were authored into `.claude/skills/` (`paper-read`, `paper-qa`, `citation-check`) so the agent's literature behavior ships with the repo.

2. Where the agent was particularly useful

Scaffolding and API plumbing. The four-source search bundle (Exa keyless MCP + Semantic Scholar + OpenAlex + arXiv, title-dedupe merge, per-source `ok/limited/error/skipped` status, 7-day cache, 8 s per-source timeout) and the SQLite + FTS5 library layer were generated correctly on the first pass, including the OpenAlex `abstract_inverted_index` decoder — a format detail I would have missed by hand.

Paper-pilot design transfer. I asked the agent to study `aytzey/paper-pilot` (MCP paper server) and port only its ideas to TypeScript: explicit failure (`sourceStatus` + `retryable` instead of silent `catch(()=>[])`), paged read cursors (`/api/read` with `nextOffset`), and honest extraction states (`text_extracted/partial_text/no_text`). The agent kept the semantics while writing original code — no Python was introduced, per my constraint.

UI state machine. The five-machine frontend (search/library/upload/reader/stream), rAF-batched SSE appends, chunked card mounting, and the `[p.X]` citation-pill renderer were iterated through screenshot-driven loops. The agent applied each visual fix (citation/link style split, `nowrap` pills, `ellipse()` titles) without regressing behavior.

Configuration semantics. Key management follows omp conventions: project `.env` (never committed, `.gitignore`-guarded), `/api/settings` reporting only modes (`proxy-managed`/`keyless-mcp`, never values), and the Agent-pane Keys panel that previews `.env` lines for manual paste instead of POSTing secrets. Default inference is `deepseek/deepseek-v4.1-flash` via local proxy.

3. Where the agent failed, and how I corrected it

Wrong-paper backfill (worst failure). When arXiv rate-limited BERT metadata, the agent added an OpenAlex fuzzy fallback that matched `1810.04805` to an unrelated hate-speech paper and wrote it into the library as BERT. I caught it by reading the fetched title, deleted the row, and removed the fallback entirely: the rule is now "filename placeholder + `metaMiss` flag beats a guessed title." Lesson: fuzzy identity resolution must be exact-ID or nothing.

Title hallucinations from body headings. The H1 fallback scanned the whole document, promoting `Input-Input Layer5` (a section head) to the title. Fix: only the first 3–5 headings count, watermark lines (`arXiv:…`) excluded. Similarly, the excerpt cutter stopped at `<!-- Page 1 -->` (cut = 0) until comment lines were skipped, then overshot into author emails until `@`-lines were skipped — three iterations to land on `#### Abstract`.

Boilerplate blindness. arXiv cover lines (`Provided proper attribution…`, `reproduce the tables…`) leaked into cards and LLM context across four fix rounds, because each line had a different markdown prefix (`####` vs bare). Fix: prefix-tolerant regexes applied server-side so cards and prompts start at content.

Silent 404 on ask. Search results lived only in a memory map the server didn't consult, so asking about an unsaved paper returned 404 swallowed by `EventSource.onerror`. Fix: `seenPapers` registry (2000-entry LRU) consulted by `/api/ask` and `/api/summarize`, plus a visible error bubble instead of silence.

Stale DOM/const errors. Two edits dropped the `$` helper and broke `mountChunked` ordering; symptom was "Library click does nothing" with zero console errors (the rAF path swallowed it in headless tabs). I diagnosed it by invoking `card()` vs `mountChunked()` separately in-page, then restored the helper and made the first chunk mount synchronously.

Screenshots referenced in the report appendix show the prompt → agent diff → my correction for the backfill and title cases.

4. How much I inspected, modified, or debugged

Roughly 40% generated-as-is (search providers, DB schema, SSE plumbing), 40% directed iteration (I specified the fix, the agent wrote it: dedupe keys, prompt constraints, citation pills), 20% hand repair (the `return bundle` deletion that blanked `/api/search`, the severed `searchArxiv` header after inserting `lookupArxiv`, the `$` helper loss). Every agent edit was verified the same turn: `curl` the endpoint or screenshot the UI before moving on. No edit was accepted on description alone. The ten UI loops (v1–v10 in `PLAN.md` §11) averaged two tool calls per fix, all computer-use verified.

5. What I learned about controlling a coding agent

Constraints beat instructions. "No Python", "never POST keys", "exact-ID or nothing" produced better code than paragraphs of intent. The agent respects hard rules; soft guidance gets averaged away.

Verify in the same turn. The failures that cost the most (wrong-paper backfill, blank search) were edits I accepted without an immediate `curl`/screenshot. The cheapest policy was: every mutation gets one observation before the next mutation.

Small diffs, named files. Pointing at `lib/search.ts:101-141` with the exact failing input beat "fix the metadata lookup." The agent is precise when the target is precise and speculative when it isn't.

Keep the agent's mistakes. The `metaMiss` flag, the `seenPapers` registry, and the `.env-preview` panel all exist because something broke. Designing the failure surface (honest statuses, visible errors) mattered more than preventing every failure.

Toxic-skill hygiene is real work. Before installing any public skill I read its `SKILL.md` for `curl|sh`, `~/.ssh` reads, and env exfiltration; keys stay in `.env`, never in chat or the repo. The OWASP AST01 pattern (prose + script dual payload) was the checklist.